

Agentic Deep Research System

面试技术拷打笔记（TeX 正式版）

面向：技术面试 / 架构深挖 / 治理闭环追问 / 高压交叉盘问

版本：v2.0

日期：2026-05-27

目录

1 如何使用这份笔记	1
2 第一轮拷打：先把项目说清楚	1
2.1 先用一句话定义这个项目	2
2.2 面试开场应该怎么讲	2
2.3 系统的分层怎么看	2
2.4 为什么这个系统不是简单的 RAG 问答	3
3 第二轮拷打：为什么这么选，而不是别的	3
3.1 Agent 编排框架：为什么选择 LangGraph	4
3.2 LangGraph 和 DAG 的关系	5
3.3 为什么还要 Harness，而不是只靠 LangGraph	6
3.4 为什么要引入 Skill 体系，而不是只拼 Prompt	7
3.5 如果 Skill 规模很大，怎么避免启动慢、匹配慢和运行中断线	8
3.6 LLM 选型：为什么强调结构化输出和工具调用	9
3.7 浏览器自动化：为什么选择 Playwright	10
3.8 为什么要实现三层浏览器提取策略	10
3.9 RAG 与向量库：为什么选择 pgvector	10
3.10 为什么内部检索不是“查一下就回答”	11
4 第三轮拷打：系统到底怎么跑，状态怎么流	11
4.1 从 API 到报告的完整生命周期	12
4.2 公开状态与内部运行状态为什么要分开	13
4.3 ResearchState 为什么是这个系统的核心	13
4.4 Planner 如何生成 DAG，为什么还需要 fallback	14
4.5 DAG 的正确性如何保证	15
4.6 为什么要先 internal RAG，再决定要不要联网	16
4.7 工具节点为什么必须返回显式结果契约	16
4.8 怎样避免单点故障把整条链路拖死	17
4.9 系统会不会从历史失败里吸取教训	18
4.10 Search / Browser / RAG 三类节点如何分工	19

5 第四轮拷打：关键机制是不是你真的做过	20
5.1 Analyst 为什么单独存在	20
5.2 Reflection 为什么不能省掉	21
5.3 Reflection 和传统 reviewer 的区别	21
5.4 RRF 融合检索为什么合理	21
5.5 Checkpointer 的真实价值是什么	22
5.6 为什么 SSE 对这个系统重要	23
6 第五轮拷打：性能、成本和资源怎么扛	23
6.1 为什么并行执行能显著降低时延	23
6.2 Token 优化策略	24
6.3 数据库优化怎么看	25
7 第六轮拷打：你怎么证明它没胡来	25
7.1 我会看哪些指标	26
7.2 trace 为什么要分成几条线	26
8 第七轮拷打：哪些能吹，哪些不能吹	27
8.1 当前真实完成度应该怎么诚实表达	27
8.2 失败语义为什么是治理第一优先级	28
8.3 会话级预算治理现在到了什么程度	29
8.4 法证级审计现在到了什么程度	30
9 第八轮拷打：邪恶 Agent 怎么管住	31
9.1 治理闭环现状判断	31
9.2 审批状态源（Approval Source of Truth）	32
9.3 为什么外部审批只能是信号，不能是唯一裁决	33
9.4 Harness 外挂设计	33
9.5 MCP 安全接入原则	34
9.6 五类幻觉怎么治理	35
10 第九轮拷打：高压追问怎么接	35
10.1 为什么选择 LangGraph，而不是直接做多 Agent 对话	36
10.2 为什么动作审批的权威源必须是 DB	36
10.3 为什么外部审批不能直接作为执行依据	36

10.4 为什么要在 LangGraph 外再加 Harness	37
10.5 为什么 MCP 必须先过 policy proxy	37
10.6 预算治理为什么不能只做软提示	37
11 附录：当前仓库现实、边界与演进路线	38
11.1 当前已核实的现实结论	38
11.2 当前仍存在的关键边界	39
11.3 建议的三阶段演进	39
11.4 最终的面试总结句	39

1 如何使用这份笔记

这份文档不是项目报告

这份文档的用途不是把项目按模块介绍一遍，而是训练一种更适合技术面试的表达方式：面试官会先确认你是不是自己做过，再追问你为什么这么选，接着看你能不能把边界、缺陷、治理和演进路线讲清楚。

建议的回答顺序

回答时不要一上来就堆框架名。最稳的顺序是：先定义系统解决什么问题，再讲主链路怎么跑，再讲为什么这么选，再讲当前已经做到哪里，最后讲哪些地方还没做完、准备怎么补。这样既像工程师，也不像背稿。

使用方法

这份笔记更适合按“结论、深入分析、边界”三层来准备。先用一两句话给出立场，再补业务抽象、替代方案、代价和故障模式，最后明确哪些能力已经落地、哪些仍有边界。面试官真正想听的通常不是定义本身，而是你是否能在压力下把判断讲完整。

这份笔记的使用原则

第一，不要把设计稿说成已上线功能。第二，不要把历史 bug 说成当前仍存在的问题。第三，不要把治理愿景说成生产级闭环。第四，遇到面试官追问时，优先讲事实、边界和取舍，不要用空话兜底。

2 第一轮拷打：先把项目说清楚

这一轮面试官在试什么

这一轮不是考你会不会背名词，而是在判断你能不能用两到三分钟把系统定义准确。面试官真正想听的是：这到底是不是一个严肃的研究型系统，还是只是把搜索、RAG 和报告拼到一起的 Demo。

这一轮最常见的追问

- 这个项目一句话到底是什么，不要给我讲一堆模块名。
- 它和普通 RAG、普通搜索问答、普通 workflow 引擎到底差在哪。
- 你这个系统的输入、处理中间态、输出结果分别是什么。
- 如果我只给你两分钟，你最希望我记住这个项目的哪三个关键词。

2.1 先用一句话定义这个项目

一句话总述

这是一个以 LangGraph 为业务编排内核、以 Search / Browser / RAG 为取证能力、以 Analyst / Reflection / Report 为分析闭环、并逐步向治理型 Agent 演进的研究系统。

2.2 面试开场应该怎么讲

推荐开场

如果面试官先问“这个系统是干什么的”，最稳的讲法不是直接说“这是一个 AI Agent”，而是先把它定义成一个 `evidence-driven deep research workflow`。它的核心目标不是闲聊，不是通用办公自动化，而是围绕“搜索、取证、对比、分析、引用、报告”构造一条可重复执行的研究链路。

我通常会用下面这段话开场：

标准开场答案

这个系统的目标是把用户问题转换成一张可执行研究图。Planner 先把问题拆成 DAG，Search / Browser / RAG 节点分别去互联网、网页和内部知识库取证，Analyst 负责综合证据形成结构化分析，Reflection 负责做事实、一致性和引用覆盖率校验，最后 Report 输出带引用的研究报告。系统的设计重点不是单次回答得多花哨，而是证据链、失败语义、预算控制和后续治理扩展能力。

2.3 系统的分层怎么看

- 编排层：LangGraph StateGraph，负责节点、条件边、循环和并行。
- 能力层：Planner / Search / Browser / RAG / Analyst / Reflection / Report。
- 状态层：ResearchState、DAG、checkpoint、trace、tool audit、budget state。
- 数据层：PostgreSQL + pgvector，存文档、session、citation、tool audit、budget state。
- 交互层：FastAPI + SSE，负责创建任务、查询状态、流式推送进度。
- 治理层：guardrail、runtime status、budget breaker、approval source of truth、Harness supervisor、MCP proxy。

2.4 为什么这个系统不是简单的 RAG 问答

核心差异

RAG 问答通常是“检索一轮，再回答一轮”。这个系统做的是“先规划，再多源取证，再综合分析，再反思校验，再决定是否重规划”。它把单次回答升级成了一条带状态、带条件分支、带恢复与治理约束的研究流水线。

深入分析

这道题不能只停在“步骤更多”。面试官真正想确认的是，你做的到底是一个回答器，还是一个研究执行系统。普通 RAG 的默认假设是：问题边界比较清楚，检索一次拿到上下文后，模型就可以直接组织答案。所以它的核心矛盾是“召回不准、答案不顺”。而研究型 Agent 的假设完全不同，它默认问题可能被拆错、证据可能互相冲突、某个来源可能暂时不可用、预算可能不允许无限扩张、以及第一次答案可能需要被系统自己否掉重做。

因此两者的差异不只是模块数量，而是执行模型。简单 RAG 的主对象是“上下文窗口”，研究型系统的主对象是“任务状态机”。前者关注把哪些文档塞进 prompt，后者关注当前任务处于哪一阶段、还有哪些节点没跑、失败是可重试还是终止、是否允许继续联网、是否需要人工审批、是否还有预算继续扩张。只要把主对象换掉，系统设计就会从“检索增强回答”自然演进成“可治理的研究流程”。

这一点在面试里必须讲透，因为很多人会把 Search、Browser、RAG、Report 都堆在一起，自称做了 Agent。真正的分水岭不是工具数量，而是是否有显式状态、显式路由、显式失败语义和显式治理约束。你要把这层抽象抬出来，面试官才会认为你不是在做功能拼装。

3 第二轮拷打：为什么这么选，而不是别的

这一轮面试官在试什么

当你把系统讲清楚之后，面试官通常会立刻追问选型理由。这里最怕两种答法：一种是“社区都这么用”，另一种是“我觉得这个更先进”。正确答法必须同时包含备选方案、比较维度和最终取舍。

这一轮最容易翻车的地方

不要把“能用”讲成“唯一正确”。也不要回避被否选方案，因为真正做过工程的人一定经历过权衡，而不是从一开始就知道唯一答案。

这一轮最常见的追问

- 为什么是 LangGraph，不是 LangChain、AutoGen、CrewAI，或者你自己写调度器。
- 为什么浏览器自动化选 Playwright，不是 Puppeteer 或 Selenium。
- 为什么检索层放 pgvector，不直接用专门向量库。
- 为什么还要 Harness，说明 LangGraph 还不够，那你一开始为什么还选它。

3.1 Agent 编排框架：为什么选择 LangGraph

结论

研究任务本质上不是线性 Chain，而是带并行、条件分支和重规划循环的状态机，所以选择 LangGraph，而不是只用 LangChain 的串联式抽象。

维度	LangGraph	LangChain	AutoGen	CrewAI
核心抽象	状态机 / 图编排	链和工具集成层	对话式多 Agent	角色协作脚本
并行与 fan-out	原生 Send API	需手动拼接	可做但偏消息流	较弱
条件边	强	中	中	弱
循环与重规划	强	弱	中	弱
检查点恢复	支持	有限	有限	弱
适合研究 DAG	很适合	一般	偏实验	偏演示

选择理由

这里至少有两个可行方案。方案 A 是继续用 LangChain，把 Planner、Search、Report 串起来，再自己维护一套 if-else。优点是简单，缺点是并行、循环、恢复都会越来越别扭。方案 B 是直接把研究流程建模成一张图，让节点、边、状态、路由都显式化。最终选择 LangGraph，就是选择方案 B。

面试回答模板

LangGraph 解决的是“复杂多步骤任务如何以显式状态机方式执行”。研究型任务天然需要 DAG、并行、条件边和循环重规划，这四点 是 LangGraph 的强项。LangChain 很适合做工具和模型的胶水层，但不适合单独承担复杂业务 workflow 编排。

深入分析

这一题真正的考点不是“你知不知道 LangGraph”，而是“你有没有把业务问题正确映射到执行模型上”。如果业务天然 是线性的，比如固定三步：检索、总结、

输出，那么 LangChain 的链式抽象完全够用，甚至更轻。但研究型任务不一样，它会同时出现三类复杂性。第一类是并行性，你希望 search、browser、rag 在某些阶段同时跑，而不是人为串起来浪费 I/O 时间。第二类是条件性，你要根据 RAG 命中情况决定是否继续联网，根据 Reflection 结果决定是否 replan，根据审批结果决定是否恢复。第三类是循环性，你不能假设第一次规划一定正确，所以必须给重规划留合法路径。

如果用普通 chain 硬做这些事，短期能跑，长期会出现两个工程问题。一个问题是状态散落。你会把当前步骤、已完成节点、预算、失败次数、审批状态藏在 if-else 和局部变量里，导致一旦出 bug 很难解释“系统为什么走到了这里”。另一个问题是恢复困难。因为链式抽象默认关注的是一步接一步的推理，而不是一个可中断、可恢复、可审计的长期工作流。

所以这里选择 LangGraph，本质上不是“选一个更潮的框架”，而是承认这个系统从第一天起就是状态机问题，而不是提示词编排问题。面试时如果你能把这层抽象对齐讲出来，可信度会比单纯报框架名高很多。

3.2 LangGraph 和 DAG 的关系

追问答案

DAG 是业务层面的研究计划图，StateGraph 是执行层面的工作流图。Planner 生成的 DAG 解决“研究步骤怎么拆”，LangGraph StateGraph 解决“这些步骤怎样被调度、聚合、校验、循环和恢复”。DAG 是数据，StateGraph 是执行器。

深入分析

这里最容易犯的错，是把 DAG 和 LangGraph 讲成同一个东西。实际上它们回答的是两个不同问题。DAG 是“这次研究任务需要做哪些事、谁依赖谁”，本质上是业务计划；LangGraph StateGraph 是“这些计划节点在运行时怎么被驱动、怎么聚合结果、怎么决定下一步”，本质上是执行引擎。一个是业务数据结构，一个是控制流框架。

为什么要分开？因为业务计划和执行控制的变化频率不同。Planner 今天可能多生成一种节点类型，或者把 Web 节点拆得更细，这属于 DAG 层的变化；但超时重试、审批暂停、反思重规划、预算熔断这类机制，不应该每次都由 Planner 直接负担，否则模型输出一旦不稳定，执行层就会跟着失控。把 DAG 当数据、把 StateGraph 当执行器，实际上是在保护控制面不被 LLM 计划输出直接污染。

还有一个关键收益是可解释性。如果 DAG 是数据，你就能清晰地把“计划是否合理”和“执行是否正确”拆开分析。前者看节点设计、依赖关系、fallback 是否健全；后者看路由、失败语义、恢复、预算和审计是否可靠。面试里把这两个层次拆开，会显得你对系统建模有清晰边界感。

3.3 为什么还要 Harness，而不是只靠 LangGraph

方案	做法	优点	缺点
方案 A	生命周期、重试、审批、恢复都塞进 LangGraph	单层编排、概念简单	图会越来越重，治理逻辑和业务逻辑耦合
方案 B	LangGraph 管业务图，Harness 管 supervisor	治理边界清晰，适合长任务	多一层状态同步与恢复控制

最终选择

当前项目的长期正确方向是方案 B：LangGraph 负责业务编排，Harness 负责任务级控制面。因为你真正要治理的是“失败后怎么恢复、预算超了怎么刹车、审批后怎么恢复、worker 怎么抢租约”，这些都不是业务 DAG 最擅长表达的东西。

深入分析

这一题最容易答浅。很多人会说“LangGraph 不够，所以再加 Harness”，这句话不够严谨。更准确的说法是：LangGraph 擅长描述业务执行图，但不擅长承担完整的任务级控制面。业务执行图关心的是当前任务应该怎么做，supervisor 关心的是当前任务该不该继续做、由谁做、从哪里恢复做、预算是否允许做、审批是否允许做。两者不是强弱关系，而是职责分层关系。

如果把 supervisor 语义硬塞进图里，短期看起来只有一层系统，长期会有三个副作用。第一，图的节点语义会被污染。原本 analyst、reflection、report 这些节点是在表达业务，而不是在表达租约、暂停、恢复、超时、人工确认。第二，图会变得很难演进。因为每加一个治理策略，你都要再给图加分支、加状态、加条件，最后图就不再是业务图，而是一个混杂业务与控制的巨型控制器。第三，恢复语义会失真。任务级恢复往往需要依赖外部状态源，例如 harness 文件、worker 租约、checkpoint 版本，而这些都不是图内局部状态能优雅表达的。

所以外置 Harness 的关键价值在于分域定权：图内负责“怎么执行”，图外负责“是否继续执行、如何恢复执行”。这一点如果讲透，面试官会更容易相信你不是在“为了多一个名词而多一个组件”，而是在主动控制系统复杂度的来源。

3.4 为什么要引入 Skill 体系，而不是只拼 Prompt

方案	做法	优点	缺点
方案 A	直接把一段 Markdown 或 system prompt 拼到 Planner 前面	实现快、改动小	只影响文案，不是体系对象，无法约束工具和 session
方案 B	把 skill 解析成可注册对象，进入 session、planner、agent、tool allowlist	能真正纳入体系，可解释、可审计、可热更新	需要额外的 parser、registry、CRUD 和状态同步

最终选择

如果目标只是“给模型多一点领域提示”，方案 A 就够了；但如果目标是“把领域能力做成系统级扩展点”，就必须选方案 B。当前项目已经按方案 B 落地：skill 以 Markdown + YAML frontmatter 形式存储，启动时加载进 runtime registry，session 会自动匹配并允许手动启用/禁用，最终同时影响 Planner、后续 Agent 提示和 tool allowlist。

深入分析

这道题真正考的不是“你会不会做配置文件”，而是“你有没有把扩展能力做成一等系统对象”。如果只是把 skill 文件读出来，然后原样拼到 system prompt 里，它本质上还是一段文本，不是一个可治理对象。这样做最大的三个问题是：第一，session 不知道自己到底启用了哪些领域能力；第二，工具层不知道哪些 skill 允许哪些工具；第三，前端和审计层无法解释某次执行为什么走了这条策略。

至少有两个可行方案。方案 A 是 prompt-only，把 skill 当成额外文案。这种方式的优点是速度快、侵入性低，但它不会改变系统结构，最终也无法形成真正的 CRUD、热更新和 session 级开关。方案 B 是 registry 模式：skill 有稳定元数据、触发规则、允许工具、agent hints 和正文内容，运行时先做匹配，再把结果下沉到 session state。这样 Planner 可以按 skill 改写研究策略，Analyst / Reflection / Report 可以按 skill 加领域提示，工具层也能按 skill 收缩能力边界。这里必须选方案 B，因为“真正纳入体系”的定义，不是能被读出来，而是能影响调度、提示和权限。

为什么 skill 文件最终选择 Markdown + YAML frontmatter？因为这里同时有两类诉求：一类是机器可校验的稳定字段，例如 name、slug、trigger_patterns、allowed_tools；另一类是人类可编辑的正文说明，例如 overview、prompt、constraints。纯 JSON / YAML 对非工程用户不够友好，纯 Markdown 分节又太脆弱，frontmatter 则刚好把“结构化元数据”和“可读正文”分开。面试里如果你把这个点讲明白，面试官会知道你不是只在堆配置，而是在做一个面向运行时和面向人类编辑都可接受的能力层抽象。

3.5 如果 Skill 规模很大，怎么避免启动慢、匹配慢和运行中断线

结论

海量 skill 的最优解，不是把所有 skill 全量读进内存，而是做成一套 meta 常驻、content 延迟、匹配分层、session 固化、版本化回收 的链路。这样既能处理冷启动和检索精度，也能保证 skill 更新或卸载后，运行中的 session 不断线。

方案	做法	优点	缺点
方案 A	启动时全量加载所有 skill 正文并全量扫描匹配	实现简单	冷启动慢、内存重、热更新抖、难扩展
方案 B	meta/content 分层、候选渐进式匹配、正文按需加载、session snapshot 固化	冷启动和精度可平衡，运行时稳定	实现复杂度更高
方案 C	外部检索服务化，registry 只做路由	超大规模扩展性最好	过早分布式，运维成本高

最终选择

当前最合理的是方案 B。因为它不要求一开始就把 skill 服务化，但已经能解决三个真实问题：第一，启动时不再依赖全量正文加载；第二，session 匹配时不再对所有 skill 做重规则扫描；第三，skill 更新和卸载后，新老 session 可以稳定分流，而不会把正在运行的任务打断。

深入分析

这个问题表面是在问“怎么做大规模加载”，但真正有三个耦合点。第一个是检索精度。如果你为了快，粗筛得太狠，就会漏掉真正该命中的 skill；如果你为了准，对所有 skill 做正则、规则和正文语义扫描，又会把延迟打爆。第二个是冷启动。如果启动阶段必须把所有 skill 的全文、正则和提示都预热进内存，实例扩容和服务重启会非常慢。第三个是状态连续性。如果运行中的 session 还依赖全局 registry 的当前态，那么 skill 一旦被更新、禁用或删除，就可能让同一个 session 在执行中前后行为漂移。

所以最优解必须同时回答这三件事。第一，在存储上做 meta/content 分离。meta 层只保留匹配和控制所需的轻字段，例如 slug、tenant、enabled、priority、keywords、allowed tools、agent hint 摘要；content 层保存完整 Markdown、解析后的 prompt 片段和较大的说明文本。启动时只加载 meta，不加载正文。第二，在匹配上做渐进式披露。先按 tenant、project、scope、enabled 做硬过滤，再用关键词、tag、domain 或全文索引做粗筛，把候选集从上万压到几十；然后只对候选集做正则、复杂 trigger rule 和更精细的策略合并；最后只为 top-N 命中的 skill 加载正文。这里“渐进式披露”不是前端折叠，而是计算成本和信息量逐级展开。第三，在运行态上做 session 固化。session 创建时生成 resolved skill snapshot，把 skill_id、version、matched reason、effective tools、prompt

fragments 都固化到会话里。之后 Planner、Analyst、Report 和 tool allowlist 全都只读这个 snapshot，而不是反复回查全局 registry。

这套设计为什么能解决 skill 卸载后不断线？关键在于版本固定和延迟回收。skill 更新不是原地覆盖，而是新版本；skill 卸载不是立刻把正文硬删，而是让它不再参与新 session 匹配。正在运行的 session 因为已经绑定了特定 version 的 snapshot，所以会继续沿用原版本执行。只有当没有 active session 再引用这个版本时，系统才真正回收正文内容。这就把“新流量不可见”和“旧流量不断线”分开了。

再往深一点说，海量 skill 的最大陷阱其实不是加载，而是 prompt 膨胀。即使你匹配很快，如果最后把二十几个 skill 的长正文全拼进 Planner 和 Report 的 prompt，LLM 侧还是会失控。所以真正成熟的系统还会再加一层约束：只让 top-N skill 进入执行，或把同类 skill 合并成摘要，并给 skill 注入本身设置 token budget 上限。面试里如果能把“匹配快”和“提示可控”一起讲出来，可信度会高很多。

3.6 LLM 选型：为什么强调结构化输出和工具调用

选型标准

这个系统里的模型不是单纯负责写文案，它要做 DAG 生成、JSON 结构化输出、事实校验、引用覆盖率分析和报告生成，所以第一优先级不是“最会聊天”，而是“最稳定地产生结构化结果”。

维度	倾向项	原因	对本项目的影响
工具调用稳定性	高优先级	Planner / Reflection 都依赖结构化结果	降低解析失败和 fall-back 频率
中文能力	高优先级	用户查询、网页、报告以中文为主	减少跨语言语义损耗
成本	高优先级	深研究任务 token 消耗天然高	决定是否能高频运行
通用推理上限	中优先级	重要，但不是唯一约束	可由反思、检索和多轮流程补强

面试时的稳妥表述

我对模型选型的原则不是盲目追逐最强模型，而是优先看三个维度：结构化输出是否稳定、工具调用是否稳定、中文与成本是否平衡。因为在研究型 Agent 里，模型失败往往不是“文风不够好”，而是 JSON 不可解析、校验结果不稳定、成本高到没法长期运行。

3.7 浏览器自动化：为什么选择 Playwright

维度	Playwright	Puppeteer	Selenium	结论
异步模型	原生 async	原生 async	传统同步为主	Playwright 更适合 Python async 服务
自动等待	强	中	弱	减少页面交互竞态
多浏览器支持	Chromium/FF/WebKit	仅 Chromium	强	Playwright 更平衡
工程稳定性	高	中	中	对研究型抓取更合适

核心回答

我选 Playwright 的原因不是“它比较新”，而是它更贴合 Browser Use 场景。研究型浏览器节点经常面对动态加载、延迟渲染、滚动加载和反检测问题，Playwright 的自动等待、页面生命周期控制和无头执行能力更容易形成稳定工程实现。

3.8 为什么要实现三层浏览器提取策略

设计理由

不是所有页面都值得深度提取。浏览器节点如果默认把整页内容吃满，会直接推高 token 消耗、拉长时延、放大噪音。所以设计成 **snippet / skim / deep** 三层提取，本质上是把内容质量和成本一起治理。

- **snippet**: 优先抓 meta 和少量核心文本，适合快速判断页面是否有价值。
- **skim**: 抓主要段落，适合普通文章和资讯。
- **deep**: 抓更多标题、段落、列表、结构化片段，适合权威报告和高价值页面。

3.9 RAG 与向量库：为什么选择 pgvector

维度	pgvector	Qdrant	Chroma	Pinecone
部署复杂度	低	中	低	SaaS
事务一致性	强	弱	弱	弱
和业务表 JOIN	方便	较弱	较弱	不方便
全文搜索协同	强	需外部系统	弱	需外部系统
运维成本	低	中	低	高

最终选择

我选 pgvector，不是因为它在纯向量召回上一定最强，而是因为它对这个项目的整体工程复杂度最低。文档、元数据、citation、research session、全文索引和向量索引都在同一套 PostgreSQL 里，事务一致性和数据联查能力非常强。

深入分析

这里的关键不是“向量数据库哪个好”，而是“这个项目最需要优化的到底是哪一层”。如果你的系统是一个高吞吐、超大规模、纯向量召回优先的检索系统，那专用向量库可能更合理。但当前这个项目不是以 ANN 性能极限为第一目标，它更像一个研究 workflow 系统。研究 workflow 系统最常见的痛点不是“向量召回每秒少了几百次”，而是“文档、会话、引用、审计、预算、审批这些状态分散在多套存储里，最后查一次完整证据链要跨很多系统”。

pgvector 的优势正体现在这里。第一，它允许文档正文、结构化元数据、全文索引、向量索引、session 记录共库，从而把一致性问题压到最低。第二，很多研究场景并不是只靠向量召回，往往还要混合关键词搜索、过滤 group/source、回查 citation 元数据，PostgreSQL 在这些组合查询上非常顺手。第三，运维心智负担更低。对于一个还在快速演进治理闭环的系统来说，减少基础设施种类往往比追求单点最优性能更重要。

这题如果只回答“因为部署方便”会太浅。更好的讲法是：我不是在做一个独立的向量检索产品，我是在做一个带审计、带引用、带任务状态的研究系统，所以我优先优化的是系统整体复杂度，而不是某一个检索子模块的局部极值。

3.10 为什么内部检索不是“查一下就回答”

核心区分

内部 RAG 在这个系统里不是回答器，而是证据源。它负责给出可引用片段、提供内部知识背景、帮助决定是否还要联网搜索，但最终回答仍然要经过 Analyst、Reflection 和 Report 链路。

4 第三轮拷打：系统到底怎么跑，状态怎么流

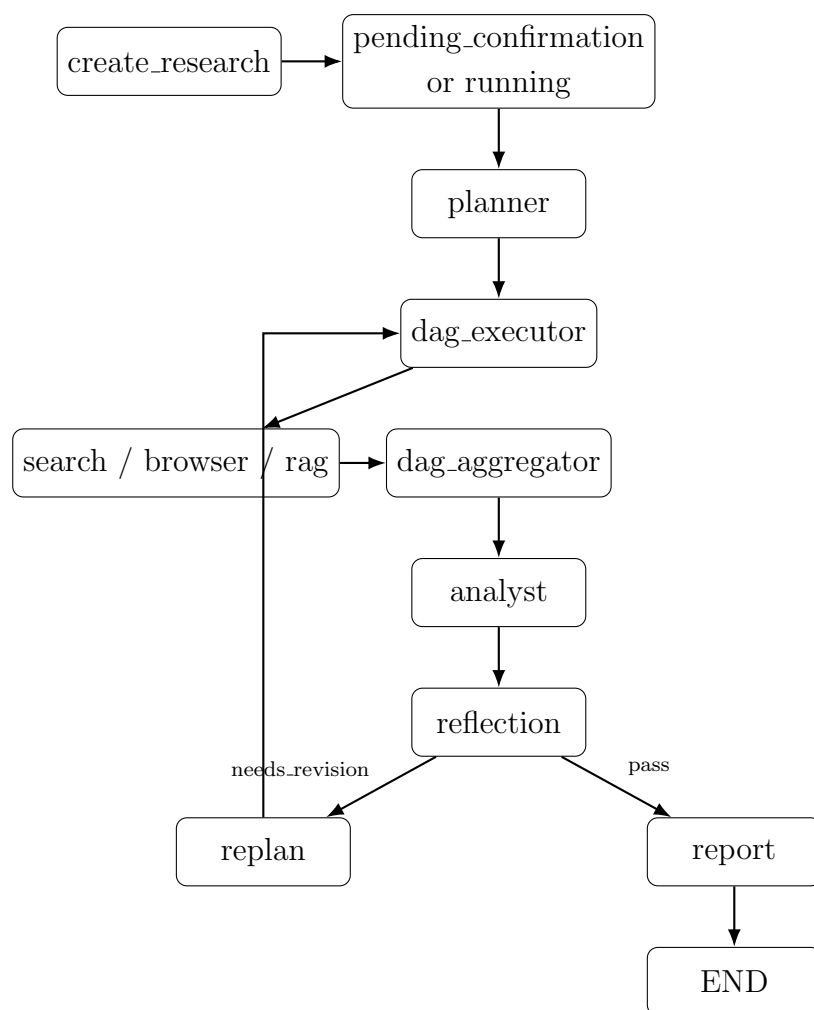
这一轮面试官在试什么

这一轮会从“你为什么这样设计”切到“这个系统运行时到底发生了什么”。如果你只能讲模块名，讲不清状态怎么变化、节点怎么路由、失败怎么传播，面试官会判断你更像读过代码，而不是做过系统。

这一轮最常见的追问

- 一个任务从 API 创建到报告产出，中间经过哪些确定的节点。
- Planner 产出的 DAG 和 LangGraph 本身是什么关系。
- public status 和 runtime status 为什么要分开，不分会出什么问题。
- 工具节点失败以后，状态怎么传给聚合器，聚合器怎么决定下一步。

4.1 从 API 到报告的完整生命周期



现状说明

当前生命周期是：create_research 建 session，状态先进入 pending_confirmation 或 running，随后后台运行 LangGraph: planner -> dag_executor -> search/browser/rag -> dag_aggregator -> analyst -> reflection -> replan/report -> END。这条主链路已经打通，是系统最核心的业务骨架。

4.2 公开状态与内部运行状态为什么要分开

核心设计

对外 API 的状态要稳定，对内运行状态要足够细。公开层只需要 `pending / running / completed / failed` 这种粗粒度语义，但治理层必须区分 `awaiting_approval / retryable_failed / terminal_failed / completed`，否则失败恢复、审批暂停和预算熔断都没法精确表达。

4.3 ResearchState 为什么是这个系统的核心

Listing 1: ResearchState 中最关键的状态维度

```
task_id / user_query / status / session
dag / current_executing_nodes / completed_nodes
tool_histories / node_outcomes / collected_evidence
verification / revision_needed / revision_count
runtime_status / budget_state / pending_approvals
agent_trace / guardrail_trace / errors
```

设计原则

ResearchState 不是简单的“共享上下文容器”，而是整个工作流的单一运行时状态机载体。凡是会影响路由、恢复、预算、审批和审计的字段，都应该显式出现在 state 里，而不是藏在零散的局部变量中。

深入分析

ResearchState 这一题本质上是在回答“系统的 source of execution truth 在哪里”。如果个工作流系统没有统一的运行时状态载体，通常会出现三类问题。第一类问题是节点之间靠隐式约定通信，比如某个节点默认下游一定会去读某个临时变量或者某个 trace 片段，这会让路由和恢复逻辑变得脆弱。第二类问题是状态不完整，表面上流程能跑，但到了出错、回放、审计的时候才发现缺关键字段。第三类问题是治理字段没有一等公民地位，预算、审批、失败分类、runtime status 这类字段被塞进 `review_status` 或普通文本里，后面很难做严肃的控制和恢复。

把这些状态提升到 ResearchState 层有一个很重要的工程收益：你可以把“路由条件”“恢复条件”“审计快照”“预算熔断”都建模为状态上的显式变换，而不是 if-else 里的旁路副作用。这样做的代价当然也有，就是 state 会逐步变大、会需要整理 merge 语义、会需要对哪些字段累加、哪些字段覆盖有严格约束。但对于一个长期任务系统来说，这个代价是值得付的，因为它换来的是可解释性和可恢复性。

4.4 Planner 如何生成 DAG，为什么还需要 fallback

面试回答

Planner 的职责不是直接给最终答案，而是把用户问题拆成一张可执行研究图。它会让 LLM 生成 DAGDefinition，里面包含节点类型、查询内容、依赖关系和并行性。但因为 LLM 可能输出坏 JSON、空结果或不合理节点，所以必须准备 fallback DAG，保证流程至少能退化成“搜索 + 浏览 + 内部检索 + 分析”这条保底路径。

深入分析

Planner 这一题考的不是“会不会让模型吐 JSON”，而是“你有没有把 LLM 放在一个可失败的位置上”。如果把 Planner 产物当绝对真理，系统会在两个地方变脆。第一，结构脆弱。模型可能输出坏 JSON、漏依赖、生成不存在的节点类型，或者把本该并行的步骤串起来。第二，策略脆弱。即便 JSON 结构合法，它也可能给出一个业务上很差的计划，比如过度依赖外网、把低价值网页抓取得过深、或者在证据不足时就安排报告节点。

所以 fallback 的意义不是给 Planner 擦屁股，而是承认 Planner 是智能组件，不是可信编译器。工程上至少有两个方案。方案 A 是 Planner 失败就让整个任务失败，优点是语义简单，缺点是把整个系统可用性绑死在一次模型输出上。方案 B 是 Planner 尽量生成高质量 DAG，但执行层永远保留一条保底研究路径。这里显然要选方案 B，因为研究系统的首要目标是“任务可继续”，而不是“计划一定漂亮”。

更深一层看，fallback 还承担了平台语义稳定器的角色。它把最低可执行能力固定成一套可预测的研究骨架，使得即便未来换模型、换 prompt、换节点模板，系统也不会因为 Planner 一次抖动就彻底失能。面试时如果你能把 fallback 讲成“系统韧性设计”，而不是“异常处理补丁”，说服力会强很多。

如果面试官追问“未知问题下灵活性会不会为零”

不会。这里的灵活性不是靠预先写死所有备选方案，而是靠三层机制叠加出来的。第一层是 Planner 动态生成 DAG，而不是固定流程。第二层是 Planner 失败时有 fallback DAG，保证任务不会因为一次模型输出坏掉就整体失能。第三层是 Reflection 和 replan 形成闭环，如果第一次计划覆盖不够或者执行中暴露出失败模式，系统会带着失败记忆重新规划，而不是机械重跑旧图。

原因

很多人会把“有 Planner”误解成“已经有动态规划能力”，这不够准确。真正的动态规划能力至少要回答三个问题。第一，第一次计划失败怎么办；第二，执行到一半发现路走错了怎么办；第三，同样的错误再次出现时，系统会不会还按原路硬撞。如果这些问题答不上来，未知问题下的灵活性确实会很低。

选型

这里至少有两个方案。方案 A 是固定流程加局部兜底，例如 Search 失败就换 Browser，或者 Planner 失败就直接终止。优点是实现简单，缺点是备选路径是预写死的，面对未知问题很快失效。方案 B 是 Planner 动态生成 DAG，执行层保留 fallback DAG，质量层再通过 Reflection 和 replan 形成带记忆的闭环。这里必须选方案 B，因为研究型任务的关键不是提前枚举所有备选方案，而是让系统在运行中根据失败和证据情况重组路径。

如何做

当前这套实现里，第一次计划失败会退化到 fallback DAG，保证最小可执行能力不丢。执行到一半如果 Reflection 认为证据不足、引用覆盖不够或者整体置信度低，系统不会只把失败节点重置成 pending，而是会进入 replan，重新调用 Planner 生成一张新 DAG。更重要的是，replan 不是在真空中发生的，它会把本会话已经出现过的失败模式整理成 failure memory 注入新的规划提示，明确告诉 Planner 哪些工具模式、查询模式或错误类别已经踩过坑，优先换工具、缩小查询范围或改变取证路径。

效果

这样处理以后，系统的灵活性不再依赖预先写死多少分支，而是依赖三种可组合能力：动态拆图、保底退化、带记忆重规划。它解决的不是“Planner 永远正确”，而是“Planner 不正确时任务还能继续，而且第二次规划会比第一次更少重复犯错”。

4.5 DAG 的正确性如何保证

- 数据结构层：Planner 输出被解析成 PlanNode / PlanEdge / DAGDefinition。
- 执行层：get_executable_order() 用拓扑思路分批求可执行节点。
- 容错层：非法节点会被过滤，Planner 失败时回退到 fallback DAG。
- 策略层：系统还会强制内部 RAG 先于 Web 节点执行，形成 internal-first 检索策略。

深入分析

DAG 正确性不能只理解成“有没有环”。在这个系统里，它至少有三层含义。第一层是结构正确，即节点类型合法、依赖存在、拓扑可排序。第二层是执行正确，即调度器不会把失败节点当成完成，也不会在依赖未满足时提前放行下游。第三层是策略正确，即图虽然形式上可跑，但是否符合平台要求，例如 internal-first、预算优先、审批前不得执行高风险写操作。

这也是为什么单靠 Planner 输出校验不够。很多人会说“把 JSON schema 校验

做好就行”，这只能保证结构层。真正难的是执行层和策略层，因为这些正确性是运行时才能暴露的。比如某个 Browser 节点虽然结构合法，但它连续超时后是否应该进入 retryable failed，还是被预算 breaker 直接跳过；某个 MCP 写工具虽然在图里合法，但如果缺审批，运行时就必须被挂起。这些都不是静态 schema 能解决的问题。

所以这里的正确性设计本质上是“静态约束 + 动态约束”的组合。静态部分让 DAG 至少能被解释，动态部分保证图在真实世界里按治理要求运行。面试里如果你能明确这两层，说明你理解的是工程正确性，而不是算法课里的 DAG 定义。

4.6 为什么要先 internal RAG，再决定要不要联网

设计理由

研究型系统不应该把互联网搜索当作唯一真相源。内部知识库通常更贴近企业语境，也更可控，所以系统会优先执行内部 RAG，再根据命中情况和策略判断是否继续走 Web。这样既减少无意义联网，也能降低把内部问题完全外部化的风险。

4.7 工具节点为什么必须返回显式结果契约

Listing 2: NodeOutcome 契约

```
NodeOutcome = TypedDict("NodeOutcome", {
    "node_id": str,
    "tool_call_id": str | None,
    "tool_name": str,
    "status": Literal[
        "success",
        "retryable_error",
        "terminal_error",
        "awaiting_approval",
        "skipped",
    ],
    "error_category": str | None,
    "error_message": str | None,
    "retry_count": int,
    "tokens_used": int,
    "cost_usd": float,
    "result_count": int,
    "approval_request_id": str | None,
})
```

为什么不能只靠 `tool_histories`

`tool_histories` 更适合审计，不适合做唯一状态源。因为审计记录的目标是“发生了什么”，而路由的目标是“下一步怎么办”。如果失败语义只藏在 `tool history` 里，聚合器很容易在语义不明确时把节点误标成完成。显式的 `NodeOutcome` 正是为了解决这个问题。

深入分析

这道题是整个治理改造里最有代表性的一个点，因为它直接说明“记录”和“控制”不能混为一谈。`tool_histories` 天然 是面向审计的结构，它关注的是调用了什么工具、传了什么参数、耗时多久、结果摘要是什么。它当然很重要，但它不是为路由设计的。路由真正需要的，是一组最小而明确的控制语义：这次调用是成功、可重试失败、终止失败、等待审批，还是被跳过；如果失败，失败类别是什么；如果等待审批，审批请求 ID 是什么。

如果没有 `NodeOutcome`，聚合器通常只能从一堆宽泛的历史记录中“推断”当前节点是不是算完成。这种推断最危险的地方在于，一旦工具层记录不完整或者语义不稳定，执行层就会把错误状态误判成完成状态。历史上那个“工具失败但节点被标 done”的 bug，本质上就是因为控制语义没有一等建模。

所以 `NodeOutcome` 的价值不在于字段好看，而在于它把执行层真正需要的控制信息抽了出来。你可以把它理解成工具层和调度层之间的一份契约：工具层负责把业务结果和失败语义翻译成一组最小控制信号，调度层再用这组信号决定下一步如何路由。只有这样，审计和控制才不会互相污染。

4.8 怎样避免单点故障把整条链路拖死

方案	做法	优点	缺点
方案 A	任意节点失败就整任务失败	语义简单	单点故障风险高，未知问题可用性差
方案 B	失败显式分类，临时故障指数退避重试，永久故障终止或重规划	可恢复性强，治理语义清晰	需要更多状态和策略维护

原因

如果系统不做错误分类，最典型的后果就是两头都错。第一类错法是把瞬时抖动当永久错误，明明一次 `retry` 就能恢复，却直接把整条链路打死。第二类错法是把永久参数错误当临时波动，结果系统拿着错误输入不断重试，把预算和 `wall clock` 一起烧掉。更进一步，如果可重试失败每次都立刻再打一次，上游正处在抖动窗口时，系统只会更快撞上限流和预算阈值。

选型

这里至少有三个可行方案。方案 A 是所有失败都统一成一个布尔值，只决定成功或失败，实现最简单，但治理语义几乎为零。方案 B 是显式区分 `retryable_error` 和 `terminal_error`，但失败后立即有限重试，优点是比方案 A 强很多，缺点是对上游抖动不够友好。方案 C 是显式失败分类，再把可恢复错误交给调度层做指数退避重试，同时把终止性错误直接停掉。这里要选方案 C，因为它既能区分错误性质，又能避免在抖动窗口里无意义地密集重试。

如何做

当前实现里，工具节点不会把所有失败都压成一个布尔值，而是显式区分 `retryable_error`、`terminal_error` 和 `awaiting_approval`。timeout、429、上游 5xx、网络抖动、临时 DB 故障会被归到可恢复类别；schema 校验失败、权限问题、审批拒绝、MCP server 不可信这类会被归到终止类别。对于前一类错误，系统不会立刻机械重试，而是根据错误类别给一个基础退避时间，再随着 `retry_count` 指数增长，计算出 `next_retry_at` 写进 failure memory，并受 `max_retries_per_tool` 约束。调度层只有在退避窗口到期后，才会把节点重新放回执行队列。对于后一类错误，节点会直接标成 terminal，不再重复消耗预算，同时失败模式会写入 failure memory，为后续 replan 提供负反馈。

效果

这样做以后，系统既不会因为一次网络抖动就把整条任务打死，也不会因为一个永久参数错误而在错误输入上疯狂重试。可恢复错误会在调度层自动退避并等待更合适的重试窗口，终止性错误会被尽快收口，避免继续烧 token、tool call 和 wall clock。

4.9 系统会不会从历史失败里吸取教训

原因

如果系统每次失败都把历史清空，就会重复踩同一种坑。对研究型任务来说，真正浪费预算的往往不是单次失败，而是同一个工具模式、同一个查询模式、同一种错误类别在一个会话里反复出现。这个问题如果不处理，重试会退化成“重复犯错”，replan 也会退化成“换个 prompt 再犯一遍”。

选型

这里至少有两个方案。方案 A 是完全不记失败历史，每次重试和 replan 都只看当前节点状态，优点是实现简单，缺点是系统没有记忆。方案 B 是先做会话级 failure memory，把本次任务里的失败模式变成控制面的一等状态，再用于重试调度和 replan 提示。这里选方案 B，而不是一上来做全局跨任务学习，是因为会

话级记忆最贴近当前控制面，也最容易验证效果和控制风险。

如何做

当前已经支持会话级 failure memory，而不是全局跨会话学习。系统会总结本次任务里哪些工具模式、查询模式和错误类别已经失败过，并把这些信息记录成结构化状态。它至少做三件事。第一，重复失败阻断。同一个工具类型加同一个查询，如果已经达到最大重试次数，或者已经被判成终止性失败，系统不会继续把它当成普通 pending 节点调度。第二，退避调度。对于可恢复错误，系统会把下一次可重试时间写进 failure memory，按指数退避窗口再入队。第三，重规划提示。系统会把这些失败模式整理成简短的 planning hints 注入下一次 replan，明确告诉 Planner：哪些模式已经失败过、哪些已经 repeat blocked、优先换工具还是缩窄查询。这里也要守住边界：当前退避是图内调度层的短等待与再入队，不是 Harness 级定时唤醒系统。

效果

failure memory 的效果不是把系统吹成“会自我进化的大模型”，而是先把失败经验转化成可执行控制。它能减少会话内重复犯错，让自动重试更克制，让 replan 更有方向感。面试里最稳的说法是：我们已经实现了会话级失败记忆，能在本次任务里做到重复失败阻断、指数退避调度和带记忆重规划；但还没有把它做成跨任务长期学习系统。

4.10 Search / Browser / RAG 三类节点如何分工

- Search：适合快速事实、数据、新闻入口、网址发现。
- Browser：适合深读文章、结构化页面提取、动态网页。
- RAG：适合内部资料、历史文档、背景知识和企业语义上下文。

面试回答模板

这三类节点不是冗余，而是故意分层。Search 负责广撒网，Browser 负责深挖高价值页面，RAG 负责内部可控证据。真正有价值的研究系统不是任何问题都扔给浏览器，而是先判断哪种证据源最值得花成本。

5 第四轮拷打：关键机制是不是你真的做过

这一轮面试官在试什么

这一轮会专门挑几个最容易暴露“只懂概念、不懂实现”的点来问，例如 Reflection 有什么用、RRF 为什么合理、Checkpoint 到底帮你解决了什么。这里需要把抽象概念落回具体执行语义。

这一轮最常见的追问

- Reflection 是不是只是让模型再看一遍自己的答案。
- Analyst 和 Report 为什么不能合并成一步。
- Checkpointer 到底恢复了什么，没恢复什么。
- SSE 只是前端体验，还是系统治理的一部分。

5.1 Analyst 为什么单独存在

核心作用

Analyst 的意义是把多源证据从“片段集合”转成“结构化分析”。Search、Browser、RAG 拿回来的是证据，Analyst 负责做事实归类、冲突识别、分析组织和初步结论形成。

深入分析

这一题真正要回答的是，为什么不能让最终 Report 一步做完。答案是证据整合和结果表达虽然都要调用模型，但它们承担的是不同职责。Analyst 关注的是“证据之间到底是什么关系”，包括哪些事实互相印证、哪些来源互相冲突、哪些结论目前证据不足；Report 关注的是“如何把已经形成的分析组织成适合阅读的输出”。把两者合并，短期能省一个节点，长期会让分析过程变成黑箱。

分开设计至少有三个收益。第一，错误隔离。若报告写得不好，你可以判断是表达问题还是前面的分析就错了。第二，治理插点更清晰。Reflection 应该审的是分析质量和引用覆盖，而不是一篇已经高度修辞化的最终文案。第三，复用能力更强。未来如果要同时输出简版摘要、详细报告、内部 memo，Analyst 产出的结构化分析可以被多个 report renderer 复用。

所以 Analyst 不是为了“多一个 Agent”，而是为了把证据解释层从最终呈现层剥离出来。面试官如果继续追问“是不是过度设计”，你就可以直接回答：只有当系统目标是一次性回答时它才显得重；一旦目标是多源证据研究、复盘和质量控制，这个拆分就是必要的。

5.2 Reflection 为什么不能省掉

核心回答

如果没有 Reflection，整个系统就会退化成“拿到一些证据后直接写报告”。Reflection 的职责是把回答质量结构化成 `overall_confidence`、`citation_coverage`、`needs_revision` 等信号，再由图路由决定是否重规划。它不是装饰，而是把“能不能继续往前走”显式化。

深入分析

Reflection 的关键价值，是把“质量判断”从人脑直觉变成系统信号。没有它，系统虽然也能输出报告，但整个链路没有一个地方正式回答“这份报告到底靠不靠谱、证据覆盖够不够、是否需要再搜一次”。很多 Demo 都止步在这一步，因为只要模型能写出看起来像样的报告，就误以为流程闭环了。实际上那只是生成闭环，不是质量闭环。

备选方案至少有两个。方案 A 是完全不做 Reflection，相信 Analyst 和 Report 已经足够。优点是链路短、成本低，缺点是系统没有内生的质量回路。方案 B 是加入一个专门的反思节点，把答案质量、引用覆盖率和修订建议结构化，再把它转成路由条件。这里必须选方案 B，因为研究系统的价值不只是“能写”，而是“在证据不足时知道自己该停下还是重来”。

当然，Reflection 也不是银弹。它本身仍依赖模型判断，所以它给出的不是绝对真相，而是一个可操作的质量信号层。这一点要诚实讲清楚。它不是替代人工 reviewer，而是在自动链路里尽量提前发现低置信或低覆盖的问题。这样表达既不会夸大能力，也能体现你理解它的工程位置。

5.3 Reflection 和传统 reviewer 的区别

- Reviewer 更偏规则门禁或人工审查。
- Reflection 更偏模型辅助的自校验与质量评分。
- 在当前项目里，Reflection 和动作审批 reviewer 已经分开：Reflection 负责答案质量自校验，approval reviewer 负责高风险动作的暂停、审批和恢复入口。

5.4 RRF 融合检索为什么合理

Listing 3: RRF 的直观公式

```
score(d) = sum(1 / (k + rank(d)))
```

设计理由

RRF 不是为了追求算法花哨，而是为了在多路检索结果之间取得稳定折中。向量相似度、关键词匹配、重排结果的分值尺度往往不一致，直接线性加权很脆弱；RRF 只依赖 rank，更稳、更简单，也更容易工程落地。

深入分析

RRF 合理，不是因为它在论文里很经典，而是因为它非常适合工程环境里的“异构召回”。在真实系统里，向量检索、关键词检索、规则检索、重排模型往往来自不同模块，它们的分值空间根本不可直接比较。你如果做线性加权，就必须不断调分值归一化、权重比例、场景偏置，这在数据分布变化时会很脆。

RRF 退一步，只相信名次，不相信原始分数。这样做牺牲了一些极致调优空间，但换来的是稳定性和可维护性。对于一个研究型系统来说，这通常是正确取舍，因为它需要一个在数据源变化、召回器变化时依然不至于失控的融合策略，而不是一个只在离线集上分数最好、上线后难维护的复杂公式。

如果面试官追问为什么不用学习排序或者更复杂的融合，你可以直接回答：可以做，但前提是你已经有足够稳定的数据闭环和标注反馈。当前阶段更值得优先的是稳健和可解释，而不是把排序问题过早做成重机器学习工程。这个回答会显得很务实。

5.5 Checkpointer 的真实价值是什么

核心答案

`thread_id=session_id` 保证的是会话隔离，Checkpointer 提供的是状态恢复能力。它能让 LangGraph 在节点边界保存执行快照，使流程在中断后有机会从已知状态继续。但要注意，检查点本身不是完整治理方案，它不自动解决预算、审批、审计和 supervisor 恢复控制。

深入分析

这一题最常见的误区是把 checkpointer 神化，好像一旦有了 checkpoint，恢复问题就全解决了。实际上 checkpointer 解决的是“图内状态快照”问题，不是“任务级治理闭环”问题。它很擅长保存某个 thread 在某个节点边界上的状态，让你不必每次都从头开始跑；但它不天然知道当前 worker 是否还持有租约，不知道预算是否已超，不知道某个动作是否还在等待人工审批，也不知道 supervisor 现在是否决定暂停整个任务。

所以 checkpointer 的正确定位是：它是恢复机制的重要组成部分，但不是完整的恢复控制面。你可以把它理解成恢复所需的“状态底座”，而 Harness 和数据库里的 runtime/approval/budget 状态则是在定义“到底能不能恢复、应该恢复到哪里、恢复后是否继续执行”。面试时如果你能把 checkpointer 的能力边界讲清

楚，反而更显得你做过真实长任务系统，而不是把框架能力误当成系统能力。

5.6 为什么 SSE 对这个系统重要

核心理由

研究任务天然就是长任务，如果没有流式可观测性，用户只能看到一个很长的 loading。SSE 的价值不是“酷炫实时输出”，而是把 agent trace、tool 进度、状态变化、报告片段和错误事件透明推给前端，让任务变成可感知、可追踪、可中断观察的过程。

6 第五轮拷打：性能、成本和资源怎么扛

这一轮面试官在试什么

很多候选人在这里会只讲“做了并发、做了缓存、做了优化”。这不够。研究型 Agent 的资源问题至少包含三类：时延、token/cost、数据库与工具开销。面试官想看的是你有没有把它们视为同一个系统问题。

这一轮最常见的追问

- 这个系统最主要的耗时在什么地方，是模型、检索还是网页抓取。
- 你怎么避免 Browser 节点把 token 直接吃爆。
- 预算治理是软提示还是硬熔断，熔断之后系统怎么收口。
- 为什么数据库仍然选 PostgreSQL，而不是拆成更多专用组件。

6.1 为什么并行执行能显著降低时延

Listing 4: LangGraph Send API 的作用

```
dag_executor -> [Send("search"), Send("browser"), Send("rag")]
```

面试回答

如果 Search、Browser、RAG 串行执行，总时延接近三者之和；并行 fan-out 后，总时延更接近三者中的最大值。这类任务的瓶颈往往在 I/O，而不是 CPU，所以并行调度的收益很高。

深入分析

并行执行的收益不能只用一句“总时延取最大值”带过，面试官真正想看的是你是否知道什么场景适合并行、什么场景不适合。这个系统里 Search、Browser、RAG 的共同特点是高度 I/O 密集，而且很多时候相互独立。Search 等网络返回，Browser 等页面加载，RAG 等数据库检索，这些等待时间如果串行堆起来，就是纯浪费。所以把它们 fan-out，本质上是在回收等待时间。

但并行不是越多越好。第一，外部依赖会有限流和配额，Browser 开太多页面可能直接把系统拖死。第二，并行会放大预算消耗，如果 Planner 过度拆图，虽然时延下降了，但 token、工具调用次数和外部 API 成本会同时上升。第三，结果聚合会更复杂，失败语义、超时处理和 partial result 策略都必须更严谨。

所以正确回答不是“并行一定更快”，而是“在 I/O 主导、节点相互独立、预算允许的前提下，并行能显著降低 wall clock；但必须配合批次调度、预算 breaker 和失败聚合语义，否则它只是把串行浪费变成并行失控”。这句话会让面试官感觉你真的踩过并发系统的坑。

6.2 Token 优化策略

- 输出长度分 short / medium / long 三档，不只是影响报告 token，也同步影响搜索次数、RAG 返回量和 citation 上限。
- 浏览器内容用 snippet / skim / deep 控制页面读取粒度。
- RAG 侧采用候选召回 + 排序 + 截断，避免把低价值片段灌入上下文。
- 最终报告阶段再根据预算控制输出规模，避免前面省下来的 token 最后一次性烧光。

深入分析

Token 优化真正难的地方，不在于“少输出一一点”，而在于你不能为了省 token 把证据链和答案质量一起省没了。研究型 Agent 和普通聊天模型的最大区别在于，很多 token 消耗不是出在最终回答，而是出在前面的证据获取和中间分析。网页抓取太深、RAG 返回太多低质量片段、Planner 把 DAG 拆得太碎、Reflection 重复审查太多次，都会在最终报告生成前就把预算吃掉。

所以这里的优化必须是系统性的。第一层是输入压缩，即在进入模型前先控制证据粒度，例如浏览器的 snippet/skim/deep、RAG 的候选截断、citation 上限。第二层是路径裁剪，即并不是所有任务都值得走完整 Web 扩展链路，internal-first 的策略就是一种成本治理。第三层才是输出控制，即最终报告的长度、引用数量、展开程度。把这三层分开讲，面试官会更容易相信你理解的是“成本结构”，而不是只有“把 max_tokens 调小”这种表层操作。

6.3 数据库优化怎么看

- 文档层：向量索引承担 ANN，全文索引承担关键词检索。
- 业务层：research session、citation、tool audit、budget state 都在同一库里，降低同步复杂度。
- 可观测层：指标与 trace 可以按 session 维度做统计和聚合。

深入分析

数据库选型这题不能只答“PostgreSQL 通用、成熟”。真正的判断是：当前系统的复杂度是否值得拆成多套专用存储。至少有两个方案。方案 A 是 PostgreSQL 一库承载业务数据、向量检索、审计和预算状态，优点是事务边界清晰、查询方便、运维成本低，缺点是向量能力和高吞吐日志能力不如专用系统。方案 B 是把向量库、OLTP 库、日志库拆开，优点是各组件可以针对性优化，缺点是同步、幂等、一致性和运维复杂度都会上升。

当前阶段选方案 A 是合理的，因为系统最主要的矛盾还不是极致吞吐，而是治理闭环和状态一致性。审批、预算、runtime status、tool audit 这些关键控制数据如果分散在多种存储里，恢复和审计会先变复杂。把它们尽量放在同一个事务型主库里，反而更利于把控制面做稳。

当然，这个选择有边界。随着文档规模、检索 QPS、审计写入量上升，未来完全可能拆出专用检索或时序/日志存储。但那应该建立在当前单库模型已经成为明确瓶颈之后，而不是一开始为了“架构高级感”就过度拆分。面试时把这一层演进逻辑说清楚，会比直接喊“微服务 + 多存储”成熟得多。

7 第六轮拷打：你怎么证明它没胡来

这一轮面试官在试什么

会做流程不等于会做系统。面试官在这一轮通常会问：你怎么知道它真的在工作，而不是表面有输出？你怎么定位问题？你怎么区分是模型错了、工具错了、还是治理规则拦住了？

这一轮最常见的追问

- 你会盯哪些指标，而不是泛泛地说“看日志”。
- trace 为什么要拆成 agent、guardrail、tool audit 三条线。
- 出现错误时，怎么判断是模型输出坏了还是外部工具失败了。
- 这个系统能不能做法证级复盘，复盘数据现在放在哪。

7.1 我会看哪些指标

维度	代表指标	作用
工作流层	成功率、P95 时长、重规划次数	看流程稳定性
工具层	tool success rate、平均时长、错误分类	看节点可靠性
质量层	citation coverage、overall confidence、hallucination rate	看答案质量
成本层	used tokens、used cost、tool calls、wall clock	看预算消耗
治理层	approval count、budget stop count、terminal failures	看可控性

7.2 trace 为什么要分成几条线

- agent_trace: 高层流程事件，适合 UI 和时间线回放。
- guardrail_trace: 安全和路由决策，适合解释“为什么被拦”。
- tool_call_audit: 单次工具调用明细，适合法证级追踪。

设计观点

把所有东西都塞进一条 trace 里，看起来简单，实际上后面既不好查，也不好给不同角色消费。高层事件、护栏决策、单次调用明细这三类数据天然就是不同层次，拆开才有意义。

深入分析

很多系统刚开始都会把“可观测性”理解成“有日志就行”，再进一步可能是“有一条 timeline 就行”。但当系统同时涉及业务流程、治理决策和外部工具时，单一时间线很快就会失效。原因不是数据量大，而是语义层次不同。产品和前端更关心用户可见的高层进度，例如 Planner 开始了、Report 生成了、任务完成了；安全和治理更关心“为什么这个动作被拦了”“为什么这个节点进入 awaiting-approval”；法证和排障则需要知道单次工具调用的参数、结果摘要、错误类别和耗时。

如果不拆线，后果通常有两个。第一，查询和展示成本会上升，因为每个消费者都得从同一条混杂日志里自己再切语义层。第二，边界会变模糊，尤其是当你想回答“这个错误到底是业务失败、治理阻断还是外部工具异常”时，很难快速切干净。

所以拆 trace 的核心不是“多存几份”，而是让不同层次的问题有不同层次的数据承载。面试时最好直接说：agent_trace 服务于高层流程可见性，guardrail_trace 服务于治理解释，tool_call_audit 服务于单次调用法证。这样表达会非常清晰。

8 第七轮拷打：哪些能吹，哪些不能吹

这一轮面试官在试什么

这轮最考验真实性。很多项目不是死在技术上，而是死在表述上。你如果把设计稿说成已经上线，或者把半成品说成生产级，懂行的面试官一追就穿。这里必须把“已做完”“已部分落地”“还在设计”分开讲。

这一轮最常见的追问

- 你现在到底能不能说自己做了治理闭环。
- 哪些能力已经在代码里落地，哪些只是详细设计。
- 历史上最关键的 bug 是什么，修复后语义有什么变化。
- 如果我现在把系统部署到生产，会先卡在哪个现实问题上。

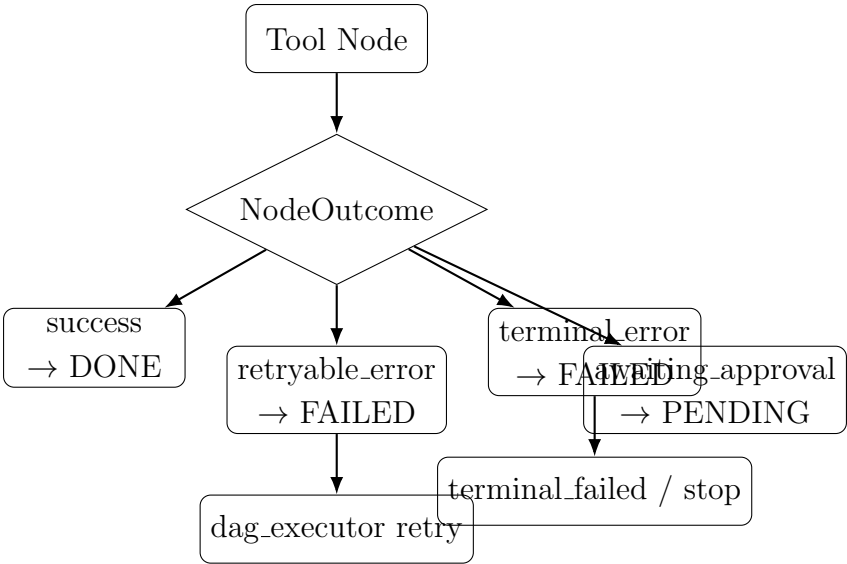
8.1 当前真实完成度应该怎么诚实表达

建议表述

当前系统已经具备研究主链路、DAG 编排、反思校验、SSE trace 和基本预算治理能力，但完整治理闭环仍在演进中。更准确的说法是：它已经是一个能工作的研究型 Agent 系统，但离严格意义上的生产级执行治理系统还有距离。

能力项	当前状态	面试表述建议
主任务生命周期	已打通	可以说“已实现主链路”
失败语义	已从历史 bug 修到显式 outcome	可以说“失败语义已显式化”
工具审计	已有独立 tool_call_audit 并改为运行中持续落库	可以说“审计已具备 crash-safe 强化版，但仍不是全量法证平台”
会话级预算 breaker	已有硬熔断与中途预算状态持久化	可以说“已支持会话级硬熔断，但 token/cost 仍部分依赖估算”
动作级审批	已接入独立 reviewer 分支与 approval API	可以说“已实现动作级审批主链路，但恢复粒度仍以批次边界为主”
Harness supervisor	已有图外 supervisor 文件协议与恢复快照	可以说“已实现最小可用任务级恢复控制，但还不是多 worker 生产调度器”
MCP 安全接入	已有 registry + policy proxy + allowlist 闸门	可以说“已接入安全通路，但默认业务图还未大规模使用 MCP 节点”

8.2 失败语义为什么是治理第一优先级



历史 bug 与当前状态

这个系统历史上最关键的问题，是工具节点失败后聚合器可能仍然把节点当成完成。这个 bug 现在已经修掉了，工具节点通过 NodeOutcome 回传显式状态，聚合器按 success / retryable_error / terminal_error / awaiting_approval / skipped 更新节点。它之所以值得在面试里重点讲，是因为它直接体现了“为什么失败语义必须显式建模”。

原因

为什么失败语义是治理第一优先级，而不是审批、预算或者 MCP？因为只要失败语义不对，后面所有治理都建立在错的状态之上。审批系统的前提是你知道当前动作到底是在成功、失败、等待审批还是已经终止；预算系统的前提是你知道当前节点是否还应该继续消耗资源；恢复系统的前提则是你知道上一次中断时到底停在了哪种失败态。历史上那个“工具失败但节点被标 done”的 bug，本质上就是完成态被污染。

选型

这里至少有两个方案。方案 A 是继续把失败信息藏在 tool history 或异常文本里，由聚合器自己猜当前节点到底算成功还是失败。优点是改动少，缺点是语义脆弱。方案 B 是让工具层显式回传一份最小控制契约，也就是 `NodeOutcome`，把成功、可重试失败、终止失败、等待审批和跳过这些状态直接告诉调度层。这里必须选方案 B，因为控制不能建立在猜测审计日志的基础上。

如何做

当前实现里，每个工具节点执行后都会显式回传 `NodeOutcome`，其中至少包含 `success` / `retryable_error` / `terminal_error` / `awaiting_approval` / `skipped` 这些控制语义，以及错误类别、重试次数、工具调用 ID 等最小信息。聚合器不再根据异常文本猜测结果，而是按这份显式契约更新 DAG 节点状态和 runtime status。与此同时，Browser 的本地 fallback 也不再被上层误记成成功页面，而是会把真实错误向上转换成 `retryable` 或 `terminal` 失败。

效果

这样做以后，失败语义从审计层附带信息升级成了调度层硬约束。工具失败不再会被吞掉，后续路由、预算治理、审批暂停和恢复决策都有了可信状态基础。面试时可以很明确地说：失败语义已经显式化，这个历史 bug 已经修掉，不应继续被当成现存问题对外表述。

8.3 会话级预算治理现在到了什么程度

原因

预算治理最容易被答成“有个 `max_tokens` 就行”，这其实远远不够。研究型 Agent 的成本不是单一来源，至少包括模型 token、外部工具调用、网页抓取时长、重试次数和整体 wall clock。只盯 token，会漏掉 Browser 一直超时、MCP 工具反复重试、或者 Planner 把图拆得过深这类真实成本问题。

选型

这里至少有两个方案。方案 A 是 observability-only，只打 warning，不改系统行为。优点是实现最简单，对现有流程侵入小；缺点是出了预算事故你只能复盘，不能止损。方案 B 是 breaker 模式，让预算阈值真正参与路由决策。比如超过某个阈值后停掉可选 Web 扩展、禁止新的 replan、必要时提前进入部分报告。这里必须选方案 B，因为治理的定义就是让规则能改变执行，而不只是描述执行。

如何做

当前项目已经不是只有长度档位了，图内已经接入 session-level budget breaker，包括 max_tool_calls、max_wall_clock_seconds，以及 max_total_tokens / max_cost_usd 的运行时累计通道。预算治理分三层。第一层是计量层，尽可能记录 usage、cost、tool call 和耗时；拿不到精确值时明确标估算，并把 estimated 与 usage_source 写入审计。第二层是策略层，在 80% 左右给 warning，在硬阈值时停止可选扩张，在更高风险时禁止 replan。第三层是收口层，即使被硬熔断，也不会让任务直接烂尾，而是跳过剩余工具节点，进入分析与部分报告，同时把预算中间态持续落到 session_budget_state。

效果

这样做以后，预算治理不再只是一个提醒系统，而是真正能改变执行路径的硬约束。系统会在预算接近危险区时发出告警，在硬阈值时主动止损并优雅降级。这里也必须诚实说明边界：当前 token/cost 通道虽然已经接到 LLM usage 和 budget state，但它仍不是绝对精确计费系统，因为上游 provider 不总返回 usage，tool 侧也未必暴露真实账单，所以系统只能做到可精确则精确，不可精确则显式标记估算。

8.4 法证级审计现在到了什么程度

当前结论

现在已经不只是 tool_histories 运行时累积状态了，系统已新增独立的 tool_call_audit 表，把单次工具调用拆成独立记录，并在 workflow 运行过程中持续 upsert。这样 session 结果查询和审计查询不再混在一行 JSON 里，中途崩溃时也不会像以前那样整批丢光。

仍未闭环的点

现在工具审计和预算状态已经前移到运行中持续落库，所以 crash-safe 能力比之前强很多。但它仍不是“无限细粒度恢复”：当前最可靠的恢复语义仍是批次边界恢复，不承诺单节点执行到一半的原地续跑。

深入分析

“法证级审计”这四个字不能乱用。真正的法证关注的是：系统在某个时刻为什么做了某个动作、当时依据是什么、参数是什么、结果是什么、谁批准的、是否经过脱敏、崩溃前最后一条可确认事实是什么。单靠最终 session JSON 或一条高层 trace，是达不到这个要求的。

当前系统之所以比早期状态前进了一大步，是因为工具调用和预算状态已经从“流程结束后统一汇总”前移成“运行中持续落库”。这意味着进程中途崩掉时，至少已经完成的调用明细、预算累计和关键状态切换不会整批蒸发。这就是 crash-safe 的核心价值，不是恢复得有多优雅，而是证据先别丢。

但也必须守住边界。现在的强项是法证记录比以前完整得多，不是说恢复已经精确到单条工具流内部。批次边界恢复依然是主语义，所以更准确的表述应该是“已经具备 crash-safe 的实时审计闭环雏形，并支持批次边界恢复”，而不是“已经完成全量执行回放平台”。这类表述差异，面试里非常关键。

9 第八轮拷打：邪恶 Agent 怎么管住

这一轮面试官在试什么

这轮不是问“你会不会做 Agent”，而是问“你敢不敢把 Agent 放进真实系统”。如果面试官开始追问审批、预算、审计、MCP、恢复、权限边界，说明他在看你有没有治理视角，而不只是能力视角。

这一轮最常见的追问

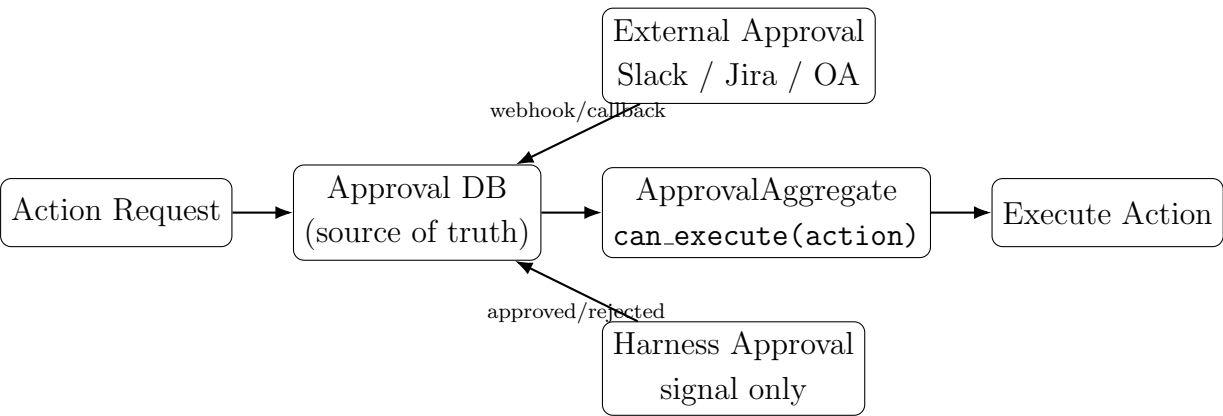
- 失败后到底是重试还是终止，谁来裁决。
- 高风险动作要不要审批，审批状态到底谁说了算。
- tool audit 能不能支撑事后法证，而不只是页面展示。
- 如果明天接 MCP 工具，为什么不能直接把它们当普通 Tool 放进图里。

9.1 治理闭环现状判断

一句话

现在系统已经有 workflow、trace、tool audit、budget breaker、动作级审批、Harness supervisor 和 MCP policy proxy 的主干治理链路，但恢复粒度、MCP 默认用量和计费精度仍有边界。

9.2 审批状态源（Approval Source of Truth）



固定立场

动作级审批的最终权威源必须是本地数据库审批单。Harness、Slack、Jira、ServiceNow 这类系统都可以作为审批信号源，但最终 `can_execute(action)` 的判断必须由本地审批聚合根负责。

深入分析

这道题背后其实是在问“source of truth 到底怎么定”。很多团队一开始会误以为，用现成平台做审批最省事，所以直接把外部状态当执行依据就行。但一旦动作进入自动化执行流，审批不再只是一个页面状态，而是一个会影响幂等、恢复、重试和危险动作下发的强控制条件。这时如果没有本地单一真相源，系统几乎必然会出现状态漂移。

为什么数据库审批聚合根最适合做最终裁决？因为动作执行发生在你的系统里，执行前最后一跳必须依赖一个你能稳定查询、能事务更新、能做版本控制、能和审计关联的本地状态。如果直接去问 Harness、Slack 或 Jira 当前是否 approved，你会立刻遇到几个问题：远端状态变化有延迟，回调可能丢失，远端 UI 和本地执行记录不在同一事务里，失败重试时也缺统一幂等点。

所以最稳的架构不是拒绝外部审批，而是重新定义它们的角色：外部系统负责承载人类交互和审批信号，本地数据库负责承载最终执行裁决。只要把这层权责讲清楚，面试官通常会认可你不是在排斥外部平台，而是在保护动作执行的一致性边界。

来源	谁是权威	优点	问题
LOCAL DB	系统自己	单一真相源、查询直观、审计强	工程量最大
HARNESS EXTERNAL	Harness 平台 Slack/Jira/OA 等	UI、RBAC、流程现成 贴合企业习惯	易和业务状态漂移 远端状态和本地状态同步复杂

Listing 5: ApprovalAggregate 的建议最小字段

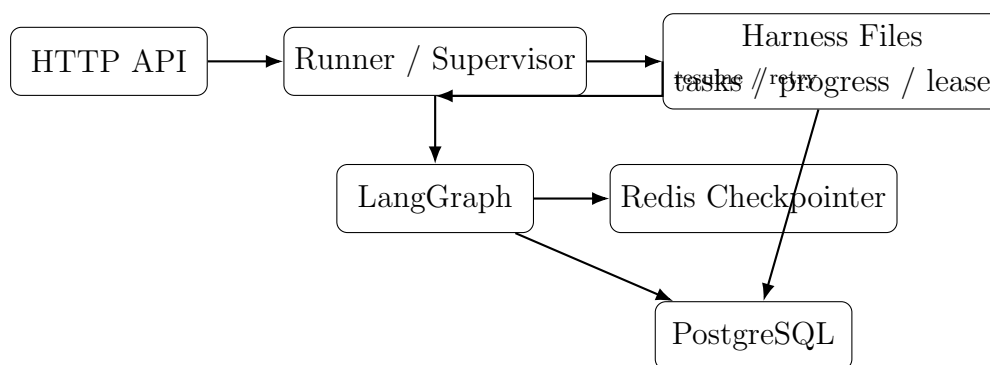
```
approval_id
action_id
approval_source
external_id
status
version
approved_by
expire_time
```

9.3 为什么外部审批只能是信号，不能是唯一裁决

原因

因为只要允许多个 source of truth 并存，就一定会遇到状态漂移：例如 Harness 已经批了，但 DB 仍是 pending；或者 Slack 上已经通过，但 agent 因为没看到本地状态变化再次发起执行。真正要避免的不是“查不到谁批了”，而是“重复执行危险动作”。

9.4 Harness 外挂设计



架构答案

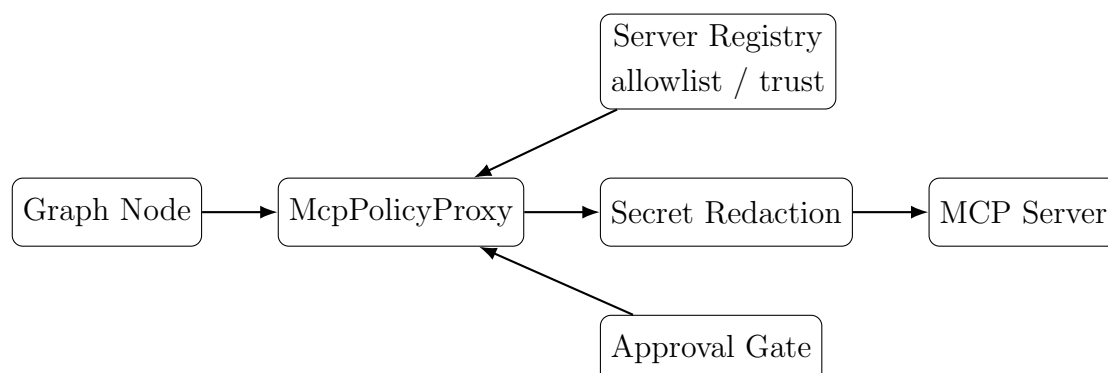
Harness 应该放在 LangGraph 外层。LangGraph 负责编排业务步骤，Harness 负责任务租约、失败恢复、暂停恢复、预算调度和 supervisor 语义。数据库负责审批、审计和预算事实，Harness 负责 lease、checkpoint 选择和 worker 协调。当前仓库已经落了最小可用实现：有 `harness-tasks.json`、progress log 和 state snapshot，但还不是完整的分布式 worker 调度系统。

深入分析

Harness 设计的关键不是“多一份文件”，而是明确任务级控制面和业务执行面的解耦边界。一个长任务系统真正难的，不是把 DAG 跑完，而是在失败、暂停、恢复、多 worker、审批等待这些现实条件下仍然能保持状态一致。Harness 文件之所以重要，是因为它把这些任务级信息从短生命周期的 HTTP 请求和进程内存里抽了出来，变成跨会话可恢复的外部上下文。

这也是为什么 progress file 和 tasks file 在这套设计里不是附件，而是恢复上下文本身。任务一旦跨会话执行，agent 的上下文窗口不再可靠，单靠口头说明或者控制台日志也不可靠。真正能支撑恢复的，是结构化任务状态、最近 checkpoint、当前 batch、审批等待点、预算使用量这些可以被下一次执行直接消费的数据。当然，这里也要诚实讲代价。外置 Harness 以后，一定会引入双写和状态同步问题，所以必须分域定权：审批、预算、审计以 DB 为准；worker lease、checkpoint 选择、恢复编排以 Harness 为准。只要把这个边界说清楚，面试官反而会认为你想清楚了复杂度，而不是机械套了一个 supervisor 名词。

9.5 MCP 安全接入原则



固定表述

MCP 是协议，不是治理系统。真正安全的接法不是把 MCP Tool 直接映射成 LangChain Tool，而是先过 McpPolicyProxy，把 server allowlist、tool allowlist、只读默认和 secret redaction 做成强闸门。当前仓库里这条代理通路和 mcp_server_registry 已经落地，但 Planner 还不会默认大量生成 MCP 节点，所以它更适合表述成“安全接入通路已存在”，而不是“业务主流量已全部走 MCP”。

深入分析

MCP 的危险点在于它天然会给人一种错觉：协议既然统一了，接上就能安全使用。实际上协议只解决了“如何发现和调用工具”，没有解决“这个工具该不该被信任、这组参数会不会越权、返回结果有没有敏感字段、写操作需不需要审批”。也就是说，MCP 统一的是接口，不是治理。

如果你把 MCP server 暴露出来的工具直接映射成普通 Tool，有三个高风险后果。第一，server 信任边界缺失。你可能根本不知道这个 server 是谁提供的、是否可信、tool 列表有没有变更。第二，能力边界缺失。同一个协议既可以暴露只读查询，也可以暴露写文件、发消息、调内部系统，如果没有 allowlist 和只读默认，agent 会天然尝试更强能力。第三，数据边界缺失。很多 MCP 返回值会带敏感字段、token、内部 ID，如果不在代理层做 redaction，这些数据会直接进入审计和上下文。

所以 policy proxy 的核心价值，是在业务图和外部工具生态之间插入一层强治理边界。它不是为了让调用更麻烦，而是为了把“能调用”变成“在规则内可用”。这一层如果讲得足够清楚，面试官通常会把你归到真正考虑过安全接入的人，而不是只会接协议的人。

9.6 五类幻觉怎么治理

- 知识幻觉：先检索，证据不足就拒答，不能拿模型记忆补业务事实。
- 工具幻觉：Observation 必须来自真实工具返回，不能让模型伪造 tool result。
- 参数幻觉：schema 校验不过就不执行，工具调用是接口，不是聊天。
- 证据幻觉：分析和报告必须能回溯到证据和 citation。
- 行动幻觉：高风险工具默认不执行，需要动作级审批门。

10 第九轮拷打：高压追问怎么接

这一轮面试官在试什么

到这一轮，面试官往往已经默认你做过一些东西了。他接下来要看的是：当问题被压得很细、很尖锐时，你能不能既守住事实边界，又把架构判断讲透。下面这些问题适合直接背熟成口头版。

高压回答原则

高压追问时，不要急着追求“说满”。最稳的做法是先给结论，再补理由，最后补边界。也就是先说“我为什么这样做”，再说“备选方案为什么没选”，最后说“哪些部分我现在还没做完”。这样能显著降低被继续追着打穿的概率。

10.1 为什么选择 LangGraph，而不是直接做多 Agent 对话

答案

多 Agent 对话更适合开放式协作，不适合强流程研究任务。研究任务要求节点、依赖、预算、审计和恢复都可显式表达，LangGraph 这种状态机编排比纯消息流更适合工程化落地。

10.2 为什么动作审批的权威源必须是 DB

答案

因为动作执行是高风险事实，不能依赖多个远端状态拼凑出来。数据库里的审批聚合根必须是唯一裁决者，否则系统会出现“远端已批、本地未落”“远端已拒、本地重试继续跑”这类危险分叉。

深入分析

这个问题本质上是在问 source of truth。动作审批如果没有单一权威源，系统迟早会出现状态漂移。比如 Harness 那边显示 approved，Slack 消息里也有人点了批准，但本地任务状态还停在 pending；或者外部系统已经拒绝，本地因为网络抖动没收到回调，任务恢复时又把动作继续发了出去。对于读操作，这类漂移已经麻烦；对于写文件、发消息、调用内部系统的高风险动作，这就是事故入口。至少有两个方案。方案 A 是直接以外部审批系统或 Harness 状态作为最终依据，本地只做被动查询。优点是接入快，缺点是幂等、审计、一致性和恢复都会依赖远端可用性。方案 B 是数据库审批单作为聚合根，外部系统只提供信号，本地统一落库后再由本地状态决定是否 can_execute。这里必须选方案 B，因为动作执行最终发生在你的系统里，责任也落在你的系统里，裁决权就不能外包。再往深一点看，DB 作权威源的意义不只是“查起来方便”，而是让审批和执行共享一个可事务化的判断平面。这样才能把 approval_id、action_id、approved_by、expired、decision_version 这些信息和执行行为绑定起来，形成真正可追责的闭环。面试时把这一层讲明白，会显得你不仅懂 workflow，还懂高风险动作系统的责任边界。

10.3 为什么外部审批不能直接作为执行依据

答案

外部审批更适合作为输入信号，不适合作为最终执行前判定。标准做法是外部系统回调统一落库，更新本地审批状态，再由本地状态决定是否允许动作执行。这样幂等、审计和重试逻辑才有单一依赖点。

深入分析

这题和上一题相邻，但重点不同。上一题强调谁是最终裁决者，这一题强调为什么远端状态不能直接驱动执行。核心原因是执行系统需要的是稳定、可重放、可幂等的本地判断，而外部审批系统通常只保证“给你一个事件”，不保证这个事件会在你最需要的时候仍然可查询、可事务绑定、可与本地恢复流程完全同步。如果直接以外部状态做执行前判断，会出现三类风险。第一是时序风险，回调到了但本地还没持久化，worker 就先继续跑了。第二是幂等风险，同一个 approve 事件重放两次，系统没有本地版本控制时可能执行两次。第三是一致性风险，外部系统后续撤销、过期或权限变更，本地没有统一聚合层就很难判断该如何处理已有任务。

所以外部审批最适合的角色是“信号源”，不是“执行开关”。你的系统应该消费这个信号，更新本地审批聚合，然后只相信本地聚合状态。这样一来，重试逻辑只看本地，恢复逻辑只看本地，审计逻辑也只需解释本地状态如何变化。对工程系统来说，这种单点依赖远比每次执行前都去问一圈外部世界稳得多。

10.4 为什么要在 LangGraph 外再加 Harness

答案

因为如果把失败恢复、预算熔断、审批暂停、worker 抢租约和任务恢复都塞进图里，图就会从“业务 workflow”膨胀成“超级控制器”。Harness 的价值在于把任务级控制面从业务级编排面剥离出去。

10.5 为什么 MCP 必须先过 policy proxy

答案

因为 MCP 只定义了如何暴露工具，不定义工具是否可信、参数是否越权、结果是否含敏感信息、危险写操作是否需要审批。没有 policy proxy，MCP 接入越多，系统越接近裸奔。

10.6 预算治理为什么不能只做软提示

答案

软提示只能告诉你“快超了”，不能真正止损。对长任务 Agent 来说，真正需要的是硬熔断加优雅降级：达到阈值后停止可选工具节点，禁止进一步 replan，最后基于当前证据输出部分报告。这才叫治理，不叫提醒。

深入分析

这道题的关键，不在于预算提醒有没有价值，而在于提醒本身不具备控制力。软提示适合人类在控制台里观察，例如“这个任务已经用了 70% 预算”；但 Agent 是自动执行系统，如果它收到提示却没有强制约束，执行路径不会自动收缩，成本照样继续增长。

从治理角度看，预算问题至少有两种处理模式。方案 A 是 observability-only，只打 warning，不改系统行为。优点是实现最简单，对现有流程侵入小；缺点是出了预算事故你只能复盘，不能止损。方案 B 是 breaker 模式，让预算阈值真正参与路由决策。比如超过某个阈值后停掉可选 Web 扩展、禁止新的 replan、必要时提前进入部分报告。这里显然要选方案 B，因为治理的定义就是让规则能改变执行，而不只是描述执行。

另外还要强调“优雅降级”这半句。硬熔断不是粗暴失败，尤其对研究任务来说，很多时候当前证据已经足够生成一个部分可用的结果。真正成熟的系统不是预算一到就直接报错，而是知道什么时候该停、停了之后怎样最小化价值损失。把这一点讲出来，会让你的预算治理显得完整，而不是只有一个 kill switch。

11 附录：当前仓库现实、边界与演进路线

附录怎么用

附录不是给面试官从头念的，而是给你自己在面试前校准边界。只要这里写的是“未完成”，面试时就不要包装成“已经具备”。只要这里写的是“设计已定”，面试时就要明确补一句“但代码尚未完整接入”。

11.1 当前已核实的现实结论

- 聚合器覆盖失败语义的历史 bug 已经修复，不应继续当作现存问题对外表述。
- tool_call_audit 已落地，工具审计不再只依赖 session JSON。
- session_budget_state 与 session_runtime_state 已落地，预算和运行时状态已从收尾汇总前移到运行中持续更新。
- session 级 breaker 已有可用硬熔断版，但 token/cost 精度仍未完全做满。
- 动作级审批已经接入独立 reviewer 分支、审批表和审批 API。
- Harness supervisor 已接入最小可用实现，支持图外任务快照和批次边界恢复。
- MCP policy proxy 与 registry 已接入，但默认业务图还未大规模依赖 MCP 节点。

- skill 体系已经落地最小可用闭环：支持 Markdown + YAML frontmatter 解析、数据库持久化、CRUD、启动加载、热更新 registry、session 自动匹配与手动启用/禁用。
- skill 已不只是 prompt 附件，而是会真实影响 Planner、Analyst / Reflection / Report 提示，以及 session 级工具 allowlist。

11.2 当前仍存在的边界

需要诚实面对的点

第一，审批虽然已经进入动作级执行流，但恢复粒度主要还是批次边界。第二，tool audit 和 budget state 已经 crash-safe 强化，但还不是全量法证平台。第三，Harness supervisor 已落地最小版本，但还没形成成熟的多 worker 调度和租约竞争体系。第四，MCP proxy 已存在，但默认 Planner 仍不以 MCP 为主要节点来源。第五，计费系统仍受上游 usage/billing 可观测性限制，不能吹成绝对精确。第六，skill 当前主要影响提示和工具边界，还没有演进到“自定义执行器 / 自定义 DAG 节点类型”这一层。

11.3 建议的三阶段演进

1. Phase 1: 失败语义显式化、tool audit 持久化、会话级 budget breaker、session budget state。
2. Phase 2: 把最小可用 Harness 演进成多 worker supervisor，并把批次边界恢复得更稳；同时把 skill 从“提示和工具约束层”继续推进到更精细的 session 策略解释和 API 治理。
3. Phase 3: 扩 Planner 的 MCP DAG 生成策略、细化 per-tool capability、接入外部审批系统信号，并评估 skill 是否需要升级成可挂载的自定义节点模板体系。

11.4 最终的面试总结句

收口

如果只看主链路，这个项目已经是一个能工作的研究型 Agent。真正让我认为它有工程价值的地方，在于它没有停留在“会规划、会搜索、会写报告”这一步，而是开始把失败语义、预算、审计、审批和外部工具边界都纳入系统设计。也正因为如此，它既能讲 Agent 能力，也能讲 Agent 治理。